CH 20: INTERVIEW MINDSET

How FAANG interviewers think • Evaluation rubric • What gets you hired or rejected

PAGE 1

1 HOW INTERVIEWERS THINK

They spend 45 mins deciding:

- **Can this person break down a problem?**
- **Can they communicate their thinking?**
- **Do they handle ambiguity well?**
- **Can they write clean production code?**
- **Do they verify their own work?**
- **Are they a Senior Engineer or an order-taker?**

The interview is a pair-programming session disguised as an exam. Your job is to make the interviewer's job easy.

2 THE 4 FAANG EVALUATION PILLARS

Pillar	What They Look For	✅ Green Flag	❌ Red Flag
Problem Solving	Pattern recognition, trade-off analysis	Clarifies before coding	Jumps straight to code
Technical Skill	Correct DS choice, clean Java code	Modular functions, good names	Monolithic spaghetti
Verification	Proactive dry run, edge case thinking	Traces manually, finds bugs	"It looks right to me"
Communication	Thinks out loud, receptive to hints	Narrates reasoning constantly	Silent 10+ min coding

3 HIRING RUBRIC SCORES

Score	Verdict	Signal
1	No Hire	Couldn't solve or communicate
2	Lean No	Partial solve, weak communication
3	Lean Yes	Solved with hints, OK communication
4	**Strong Hire**	Optimal + clean code + narrated fully

Score 4 = interviewer writes: *"Would love to work with them."*

4 5 INSTANT REJECTION TRIGGERS

❌ **Silent coding for 10+ minutes.** Interviewers can't evaluate what they can't hear. Even wrong ideas spoken out loud are better than silence.

❌ **Jumping to code before clarifying.** Interviewer: "The problem said integers — did you ask if they could be negative?"

❌ **"I'm done" without dry-running.** Submitting buggy code with confidence is worse than finding your own bug.

❌ **Arguing against interviewer hints.** When they say "what about this edge case?" — they are giving you the answer. Listen!

❌ **Using variable names like a, b, temp2, x1.** `leftPtr`, `maxSumSoFar`, `visitedNodes` signal senior-level thinking.

5 WHAT IMPRESSES FAANG INTERVIEWERS

✅ **Proactively names constraints before being asked** — "I'll assume N ≤ 10^5, so O(N log N) is fine"

✅ **Compares two approaches** — "Heap vs QuickSelect: heap is O(N log K), QuickSelect O(N) avg"

✅ **Gets interviewer agreement before coding** — "Does this approach make sense before I code it?"

✅ **Writes defensive guards at top** — null check, empty array check before main logic

✅ **Dry-runs with a specific example** — traces pointer values step by step on paper/screen

✅ **Announces follow-ups unprompted** — "If the input was a stream, I'd use a sliding window instead"

GOLDEN RULE: Every minute you speak, every constraint you clarify, every trade-off you name = signal. Silence = no signal. No signal = No Hire.

👑 MID vs SENIOR vs STAFF EXPECTATIONS

The exact behavioral differences that determine your level

1 LEVEL EXPECTATION COMPARISON MATRIX

Rubric Dimension	Mid-Level (L4 / IC4)	Senior Engineer (L5 / IC5)	Staff Engineer (L6 / IC6)
🔍 **Requirement Clarification**	Asks basic: "Is input sorted?"	Asks: constraints, scale N, edge cases, return type for invalid	Identifies hidden scale (streaming, distributed), asks about memory limits, SLA
💡 **Algorithm Choice**	Proposes first working approach	Compares 2-3 trade-offs explicitly: Heap vs QuickSelect, BFS vs DFS	Proposes optimal + discusses cache locality, concurrency, and real-world gotchas
💻 **Code Structure**	Monolithic single method, ad-hoc names	Modular helpers, Java records, descriptive names, handles overflow	Production-grade: extensible interfaces, handles all error cases, comments WHY
🧪 **Verification / Testing**	Tests happy path when asked	Proactively dry-runs edge cases: empty, single element, all-same, max N	Constructs adversarial tests: worst-case input, stress test thinking, invariant checking
🗣️ **Communication Style**	Responds to questions	Thinks out loud continuously, narrates every decision	Leads the interview gracefully, guides discussion, makes interviewer feel like a colleague
📈 **Proactiveness**	Solves exactly what is asked	Asks "what if N is 10^9?" before interviewer does	Reframes the problem: "Is this actually a shortest path on a graph of states?"
🔄 **Follow-up Handling**	Waits for follow-up question	Proactively says "a harder version of this would be..."	Presents 3 alternative approaches with trade-offs, discusses system-level implications
🏗️ **Engineering Ownership**	Implementer mentality	Takes ownership of correctness and efficiency	Thinks about team impact, API design, long-term maintainability

2 SAME QUESTION — L4 vs L5 RESPONSE

Question: "Find the k-th largest element in an array."

L4 Candidate:
"I'll sort the array and return arr[n-k]." *(begins coding immediately)*

L5 Candidate:
"Before I code — a few clarifications. Can the input have duplicates? Are elements guaranteed integers? What is the range of k — is k always valid?

For approach: sorting gives O(N log N). A better approach is a min-heap of size k — O(N log K) time, O(K) space. QuickSelect can do O(N) average but $O(N^2)$ worst-case. Given no worst-case guarantee needed, I'll use the heap approach for predictability. Does that sound good?"

3 COMMON MISTAKES BY LEVEL

L4 Mistakes (getting rejected for L5):

- Solves the easy version without checking constraints
- Never mentions time/space until asked
- Thinks "it works" = done (no dry run)
- Doesn't handle null / empty / single element

L5 Mistakes (getting rejected for L6):

- Solves correctly but doesn't proactively scale
- Doesn't anticipate follow-ups before being asked
- Presents one solution without comparing alternatives
- Good code but doesn't discuss extensibility / API design

📌 **KEY INSIGHT:** To pass as L5, demonstrate engineering ownership from the first 30 seconds. To pass as L6, make the interviewer feel they're collaborating with a peer, not evaluating a candidate.

🎯 CH 21: PATTERN RECOGNITION MASTERY

30-second recognition • Constraint mapping • Hidden keywords • Wrong pattern traps

1 THE 30-SECOND RECOGNITION SYSTEM

Step 1: Read problem → underline nouns (input type) and verbs (operation type)
Step 2: Look at N constraint → maps to max allowed time complexity
Step 3: Match keyword phrases → map to pattern family
Step 4: Recall generic template → customize for problem

CONSTRAINT → MAX COMPLEXITY MAPPING

N Constraint	Max Complexity	Pattern Hint
$N \le 20$	$O(2^N)$ or $O(N!)$	Backtracking, Bitmask DP
$N \le 100$	$O(N^3)$	3D DP, Floyd-Warshall
$N \le 1{,}000$	$O(N^2)$	2D DP, Bubble Sort
$N \le 100{,}000$	O(N log N)	Sort + Heap + Binary Search
$N \le 10{,}000{,}000$	O(N)	Two Pointers, Sliding Window, Prefix Sum
$N \le 10^{18}$	O(log N)	Binary Search on Answer, Math

2 HIDDEN KEYWORD → PATTERN MAP

If you see these words...	Think this pattern
"contiguous subarray", "max/min sum"	Sliding Window / Prefix Sum
"two numbers that sum to target"	Two Pointers or HashMap
"sorted array", "find index of"	Binary Search
"top K", "K largest/smallest", "frequent"	Min-Heap of size K
"shortest path", "fewest steps"	BFS (unweighted) or Dijkstra
"all combinations", "all subsets", "generate all"	Backtracking
"minimum cost", "optimal choice at each step"	Greedy or DP
"overlapping subproblems", "how many ways"	Dynamic Programming
"merge intervals", "schedule", "overlap"	Intervals + Sort by start
"connected components", "union", "same group"	Union-Find (DSU)
"course prerequisites", "dependency order"	Topological Sort
"word prefix", "autocomplete", "starts with"	Trie
"parentheses", "valid brackets", "daily temperature"	Monotonic Stack
"range query", "point update"	Segment Tree / Fenwick BIT

3 COMPLETE PATTERN RECOGNITION MATRIX (15 CORE PATTERNS)

Pattern	Trigger Signal	Key DS	Time	Space	Classic Problems
Two Pointers	Sorted array, pair finding, palindrome	int left, right	O(N)	O(1)	LC 15, 125, 167, 11
Sliding Window	Contiguous subarray/substring, max/min K window	int[128] freq, l/r pointers	O(N)	O(1)	LC 3, 76, 424, 567
Binary Search	Sorted space, minimise/maximise feasibility	int lo, hi, mid	O(log N)	O(1)	LC 33, 875, 410, 4
Prefix Sum	Subarray sum = k, range queries	int[] prefix, HashMap	O(N)	O(N)	LC 560, 523, 1248
Monotonic Stack	Next greater/smaller, histogram bars	Deque<Integer>	O(N)	O(N)	LC 739, 84, 85, 42
Top-K Heap	K largest/smallest/frequent	PriorityQueue min-heap	O(N log K)	O(K)	LC 215, 347, 295, 973
Tree DFS	Tree path, depth, subtree, LCA	Recursion stack	O(N)	O(H)	LC 104, 124, 236, 543
Tree BFS	Level order, row by row, zigzag	Queue<TreeNode>	O(N)	O(W)	LC 102, 199, 103
Graph DFS/BFS	Connected components, flood fill, island	boolean[] visited	O(V+E)	O(V)	LC 200, 695, 130
Topo Sort	Dependencies, DAG ordering	int[] indegree, Queue	O(V+E)	O(V)	LC 207, 210, 269
Union-Find	Grouping, connectivity, cycle detection	int[] parent, rank	$O(\alpha(N))$	O(N)	LC 547, 684, 1319
1D DP	Min/max cost, count ways, linear state	int[] dp	O(N)	O(N)	LC 70, 198, 300, 139
2D DP	Grid paths, string matching, knapsack	int[][] dp	O(N·M)	O(N·M)	LC 62, 1143, 72, 416
Backtracking	All combinations, permutations, N-Queens	List<T> path, recursion	$O(2^N)$ / O(N!)	O(N)	LC 78, 39, 46, 51
Greedy + Intervals	Merge, schedule, min arrows, activity	Arrays.sort + scan	O(N log N)	O(N)	LC 56, 435, 452, 253

4 WRONG PATTERN TRAPS — PIVOT IMMEDIATELY

Looks Like	Actually Is	Failure Signal	Pivot To
Greedy (Coin Change)	DP (Unbounded Knapsack)	Non-canonical coins → greedy fails	dp[i] = min(dp[i], dp[i-coin]+1)
DFS (Shortest Path)	BFS Level Order	DFS explores deep first, misses short	Queue + visited[] BFS
Sort (Top K)	Min-Heap size K	Sort is O(N log N), heap is O(N log K)	PriorityQueue minHeap
DP (Jump Game)	Greedy (Farthest Reach)	DP is $O(N^2)$, greedy O(N)	maxReach = max(maxReach, i+nums[i])

5 WHEN NOT TO USE — COMMON CONFUSIONS

- ❌ **Binary Search on unsorted array** — must sort first or use Two Pointers
- ❌ **DFS for shortest path** — always use BFS for unweighted shortest path
- ❌ **Greedy on Coin Change with arbitrary coins** — use DP
- ❌ **Backtracking when N > 20** — exponential, look for DP or Greedy
- ❌ **Two Pointers on unsorted array** — only valid on sorted or monotone arrays
- ❌ **Union-Find for directed graphs** — only correct for undirected connectivity

🌳 TREE + GRAPH TEMPLATE LIBRARY

All 8 Tree Templates + 11 Graph Templates — the complete reusable set

T1 DFS — PREORDER / INORDER / POSTORDER

```
// PREORDER: Root → Left → Right
void preorder(TreeNode node) {
    if (node == null) return;
    process(node);    // ← Root FIRST
    preorder(node.left);
    preorder(node.right);
}
// INORDER: Left → Root → Right (BST = sorted!)
void inorder(TreeNode node) {
    if (node == null) return;
    inorder(node.left);
    process(node);    // ← Root MIDDLE
    inorder(node.right);
}
// POSTORDER: Left → Right → Root
int postorder(TreeNode node) {
    if (node == null) return 0;
    int left = postorder(node.left);
    int right = postorder(node.right);
    return combine(left, right, node); // ← Root L
AST
}
```

💡 Preorder = serialize. Inorder on BST = sorted. Postorder = compute from leaves up.

T2 BFS — LEVEL ORDER

```
List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> q = new ArrayDeque<>();
    q.offer(root);
    while (!q.isEmpty()) {
        int size = q.size(); // FREEZE level size
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode node = q.poll();
            level.add(node.val);
            if (node.left != null) q.offer(node.le
ft);
            if (node.right != null) q.offer(node.r
ight);
        }
        res.add(level);
    }
    return res;
}
```

T3 BOTTOM-UP: HEIGHT / BALANCED / DIAMETER

```
// HEIGHT (returns int)
int height(TreeNode node) {
    if (node == null) return 0;
    int L = height(node.left);
    int R = height(node.right);
    return 1 + Math.max(L, R); // ← combine
}
// BALANCED (returns bool)
boolean isBalanced(TreeNode node) {
    return checkHeight(node) != -1;
}
int checkHeight(TreeNode node) {
    if (node == null) return 0;
    int L = checkHeight(node.left);
    if (L == -1) return -1;
    int R = checkHeight(node.right);
    if (R == -1) return -1;
    if (Math.abs(L - R) > 1) return -1;
    return 1 + Math.max(L, R);
}
// DIAMETER (global max with postorder)
int maxDia = 0;
int diameter(TreeNode node) {
    if (node == null) return 0;
    int L = diameter(node.left);
    int R = diameter(node.right);
    maxDia = Math.max(maxDia, L + R);
    return 1 + Math.max(L, R);
}
```

T5 BST — SEARCH / VALIDATE / LCA

```
// BST SEARCH
TreeNode search(TreeNode root, int target) {
    if (root == null || root.val == target) return
root;
    return target < root.val ?
        search(root.left, target) :
        search(root.right, target);
}
// VALIDATE BST (pass min/max bounds)
boolean validate(TreeNode node, long min, long ma
x) {
    if (node == null) return true;
    if (node.val ≤ min || node.val ≥ max) return
false;
    return validate(node.left, min, node.val) &&
           validate(node.right, node.val, max);
}
// LCA (Binary Tree)
TreeNode lca(TreeNode node, TreeNode p, TreeNode
q) {
    if (node == null || node == p || node == q) re
turn node;
    TreeNode L = lca(node.left, p, q);
    TreeNode R = lca(node.right, p, q);
    return (L != null && R != null) ? node : (L !=
null ? L : R);
}
```

G1 BUILD GRAPH — ALL TYPES

```
// UNDIRECTED GRAPH from edge list
List<List<Integer>> buildGraph(int n, int[][] edge
s) {
    List<List<Integer>> g = new ArrayList<>();
    for (int i = 0; i < n; i++) g.add(new ArrayLis
t<>());
    for (int[] e : edges) {
        g.get(e[0]).add(e[1]);
        g.get(e[1]).add(e[0]); // remove for direc
ted
    }
    return g;
}
// WEIGHTED GRAPH
List<List<int[]>> buildWeighted(int n, int[][] edg
es) {
    List<List<int[]>> g = new ArrayList<>();
    for (int i = 0; i < n; i++) g.add(new ArrayLis
t<>());
    for (int[] e : edges)  // e = [from, to, weigh
t]
        g.get(e[0]).add(new int[]{e[1], e[2]});
    return g;
}
// GRID UTILITIES
int[][] DIR4 = {{-1,0},{1,0},{0,-1},{0,1}};
int[][] DIR8 = {{-1,-1},{-1,0},{-1,1},{0,-1},{0,
1},{1,-1},{1,0},{1,1}};
boolean isValid(int r, int c, int rows, int cols)
{
    return r ≥ 0 && r < rows && c ≥ 0 && c < col
s;
}
```

G3 BFS + DIJKSTRA

```
// BFS (unweighted shortest path)
int bfs(List<List<Integer>> g, int src, int dst) {
    int[] dist = new int[g.size()];
    Arrays.fill(dist, -1); dist[src] = 0;
    Queue<Integer> q = new ArrayDeque<>();
    q.offer(src);
    while (!q.isEmpty()) {
        int u = q.poll();
        if (u == dst) return dist[u];
        for (int v : g.get(u)) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.offer(v);
            }
        }
    }
    return -1;
}
// DIJKSTRA (weighted)
int[] dijkstra(List<List<int[]>> g, int src) {
    int[] dist = new int[g.size()];
    Arrays.fill(dist, Integer.MAX_VALUE); dist[sr
c] = 0;
    PriorityQueue<int[]> pq = new PriorityQueue<>
((a,b) -> a[0]-b[0]);
    pq.offer(new int[]{0, src});
    while (!pq.isEmpty()) {
        int[] cur = pq.poll();
        int d = cur[0], u = cur[1];
        if (d > dist[u]) continue; // stale
        for (int[] e : g.get(u)) {
            int newDist = dist[u] + e[1];
            if (newDist < dist[e[0]]) {
                dist[e[0]] = newDist;
                pq.offer(new int[]{newDist, e
[0]});
            }
        }
    }
    return dist;
}
```

G9 UNION-FIND (DSU) — PATH COMPRESSION + UNION BY RANK

```
int[] parent, rank;
void init(int n) { parent = new int[n]; rank = new int[n];
    for (int i = 0; i < n; i++) parent[i] = i; }
int find(int x) { // path compression
    if (parent[x] != x) parent[x] = find(parent[x]);
    return parent[x]; }
boolean union(int x, int y) { // union by rank
    int px = find(x), py = find(y);
    if (px == py) return false; // already same component (cycle!)
    if (rank[px] < rank[py]) { int t = px; px = py; py = t; }
    parent[py] = px;
    if (rank[px] == rank[py]) rank[px]++;
    return true; }
```

G8 TOPOLOGICAL SORT — KAHN'S ALGORITHM

```
List<Integer> topoSort(int n, int[][] prereqs) {
    List<List<Integer>> g = new ArrayList<>();
    int[] indegree = new int[n];
    for (int i = 0; i < n; i++) g.add(new ArrayList<>());
    for (int[] p : prereqs) {
        g.get(p[1]).add(p[0]);
        indegree[p[0]]++;
    }
    Queue<Integer> q = new ArrayDeque<>();
    for (int i = 0; i < n; i++) if (indegree[i] == 0) q.offer(i);
    List<Integer> order = new ArrayList<>();
    while (!q.isEmpty()) {
        int u = q.poll(); order.add(u);
        for (int v : g.get(u))
            if (--indegree[v] == 0) q.offer(v);
    }
    return order.size() == n ? order : new ArrayList<>(); // empty = cycle
}
```

🛠️ CH 22: THE COMPLETE PROBLEM SOLVING FRAMEWORK

7-step workflow • Time budget • Clarification scripts • Full worked example

1 THE 7-STEP WORKFLOW + TIME BUDGET

Min	Step	Goal
0–3	🔍 **Clarify + Constraints**	N range, negative ints, return type, edge cases
3–7	📝 **Work Examples**	Concrete 3-4 element example, trace manually
7–12	🔨 **Brute Force + Why It Fails**	State $O(N^2)$, explain bottleneck
12–17	💡 **Optimal Insight + Pitch**	Name data structure, get interviewer nod
17–35	💻 **Write Clean Java Code**	Modular, defensive guards first
35–42	▶️ **Dry Run Edge Cases**	Empty, single, duplicates, max N
42–45	📊 **Complexity + Follow-ups**	Time/space analysis, scale question

💡 **Never start coding before minute 17.** Agreement on approach = no wasted code resets.

2 CLARIFYING QUESTIONS SCRIPT

For ARRAYS:

- "Can input contain negative numbers or duplicates?"
- "Is the array sorted? Can I sort it?"
- "What is the max value of N? Max element value?"

For GRAPHS:

- "Is the graph directed or undirected?"
- "Can it have cycles? Multiple components?"
- "Are edge weights positive or can they be negative?"

For STRINGS:

- "Only lowercase letters, or ASCII or Unicode?"
- "Is the string empty a valid input?"

For DESIGN:

- "What are the most frequent operations — get vs put?"
- "Thread-safe? Persistence? Memory constraints?"

3 FULL WORKED EXAMPLE: LC 1 — TWO SUM (L5-Level Execution)

Interviewer: "Given an integer array nums and target, return indices of two numbers that add to target."

Candidate: "Before I code — quick clarifications: Can there be multiple valid answers? Is there exactly one solution guaranteed? Can we reuse the same element? And what's the range of N and element values?"

Interviewer: "Exactly one solution. No reuse. $N \le 10^4$."

Candidate: "Got it. Brute force would be $O(N^2)$ with two nested loops — too slow for $N=10^4$ in a tight time limit.

Optimal insight: use a HashMap to store complement. For each num, check if (target - num) already exists in the map. If yes, return [map.get(complement), i]. Otherwise, store nums[i] → i. This is O(N) time, O(N) space. Does that sound good?"

Interviewer: "Yes, go ahead."

```
// GUARD: always handle edge cases first
public int[] twoSum(int[] nums, int target) {
    if (nums == null || nums.length < 2)
        return new int[]{};

    // complement → index mapping
    Map<Integer, Integer> seen = new HashMap<>
();

    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (seen.containsKey(complement)) {
            // DRY RUN: nums=[2,7,11,15], targ
et=9
            // i=0: complement=7, seen={} → mi
ss
            // i=1: complement=2, seen={2:0} →
HIT!
            return new int[]{seen.get(compleme
nt), i};
        }
        seen.put(nums[i], i);
    }
    return new int[]{}; // never reached per p
roblem spec
}
// Time: O(N) — one pass, O(1) HashMap lookup
// Space: O(N) — at most N entries in HashMap
// Edge: nums=[3,3], target=6 → works! (differ
ent indices)
```

▶️ DRY RUN TABLE: nums=[2,7,11,15], target=9

i	nums[i]	complement	seen contains?	action
0	2	7	❌ No	put 2→0
1	7	2	✅ Yes! (index 0)	return [0,1]

🔄 FOLLOW-UP QUESTIONS:

- "What if there can be multiple pairs?" → collect all, return List<int[]>
- "What if array is sorted?" → Two Pointers, O(1) space!
- "What if input is a data stream?" → Keep HashMap, process one-by-one
- "What if we need 3Sum?" → Sort + Two Pointers outer loop

💡 **Always volunteer follow-ups!** It signals Senior+ thinking.

CH 25: JAVA INTERVIEW TOOLKIT + TEMPLATE LIBRARY

Collections cheat sheet • Utility APIs • Comparators • DP + BS templates

1 JAVA COLLECTIONS MASTER REFERENCE

Collection	Key Operations	Time	Interview Use	⚠️ Gotcha
HashMap<K,V>	put, get, getOrDefault, containsKey, computeIfAbsent	O(1) avg	Frequency, complement lookup, grouping	getOrDefault not same as get+putIfAbsent
HashSet<T>	add, contains, remove	O(1) avg	Dedup, existence check, visited	add() returns false if already present
TreeMap<K,V>	floorKey, ceilingKey, firstKey, lastKey, subMap	O(log N)	Ordered range queries, calendar, skyline	NavigableMap API not in HashMap!
TreeSet<T>	floor, ceiling, lower, higher, headSet, tailSet	O(log N)	K-th element, range existence	floor/ceiling return null if not found
PriorityQueue	offer, poll, peek	O(log N)	Top-K, Dijkstra, two-heap median	Default is MIN heap! Use (a,b)→b-a for MAX
ArrayDeque	push/pop (Stack), offer/poll (Queue), peekFirst/Last	O(1)	Stack, Queue, Monotonic DS	Faster than LinkedList and Stack class
Arrays utility	sort, binarySearch, fill, copyOf, toString	O(N log N) sort	Sort + BS, initialize arrays	Use Integer.compare(a,b) not a-b (overflow!)

2 COMPARATOR / LAMBDA QUICK REFERENCE

```
// MIN HEAP (default)
PriorityQueue<Integer> minPQ = new PriorityQueue<>();
// MAX HEAP
PriorityQueue<Integer> maxPQ = new PriorityQueue<>(Collections.reverseOrder());
// Sort by second element (SAFE — no overflow)
PriorityQueue<int[]> pq = new PriorityQueue<>((a,b)→Integer.compare(a[1],b[1]));
// Sort array of arrays by first col asc, then second desc
Arrays.sort(intervals, (a,b) → a[0]==b[0] ? b[1]-a[1] : a[0]-b[0]);
// Sort strings by length, then alphabetically
list.sort(Comparator.comparingInt(String::length).thenComparing(Comparator.naturalO
rder()));
// GroupBy (anagram key)
Map<String, List<String>> map = new HashMap<>();
map.computeIfAbsent(key, k → new ArrayList<>()).add(word);
```

3 JAVA TYPE CONVERSION CHEAT SHEET

From	To	Java Code
String	char[]	`s.toCharArray()`
char[]	String	`new String(arr)`
int	String	`String.valueOf(n)`
String	int	`Integer.parseInt(s)`
char	int (index)	`c - 'a'`
int	char	`(char)('a' + i)`
List<Integer>	int[]	`list.stream().mapToInt(x→x).toArray()`
int[]	List	`Arrays.stream(arr).boxed().collect(Collectors.toList())`

4 DP TEMPLATE — MEMO + TABULATION

```
// ═══ TOP-DOWN MEMOIZATION ═══
Map<String, Integer> memo = new HashMap<>();
int dp(int i, int j, String s, String t) {
    if (i == s.length()) return 0; // base case
    if (j == t.length()) return s.length() - i;
    String key = i + "," + j;
    if (memo.containsKey(key)) return memo.get(key);
    int res;
    if (s.charAt(i) == t.charAt(j)) res = dp(i+1, j+1, s, t);
    else res = 1 + Math.min(dp(i+1,j,s,t), Math.min(dp(i,j+1,s,t), dp(i+1,j+
1,s,t)));
    memo.put(key, res);
    return res;
}
// ═══ BOTTOM-UP TABULATION ═══
// dp[i][j] = min edit distance for s[0..i-1], t[0..j-1]
int editDistance(String s, String t) {
    int n = s.length(), m = t.length();
    int[][] dp = new int[n+1][m+1];
    for (int i = 0; i ≤ n; i++) dp[i][0] = i;
    for (int j = 0; j ≤ m; j++) dp[0][j] = j;
    for (int i = 1; i ≤ n; i++)
        for (int j = 1; j ≤ m; j++)
            dp[i][j] = s.charAt(i-1) == t.charAt(j-1) ? dp[i-1][j-1] :
                1 + Math.min(dp[i-1][j], Math.min(dp[i][j-1], dp[i-1][j-1]));
    return dp[n][m];
}
```

5 BINARY SEARCH — 3 TEMPLATES

```
// ═══ CLASSIC: Find exact target ═══
int binarySearch(int[] arr, int target) {
    int lo = 0, hi = arr.length - 1;
    while (lo ≤ hi) {
        int mid = lo + (hi - lo) / 2; // NEVER (lo+hi)/2 (overflow!)
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
// ═══ LEFT BOUNDARY: First position ≥ target ═══
int lowerBound(int[] arr, int target) {
    int lo = 0, hi = arr.length;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] < target) lo = mid + 1;
        else hi = mid;          // ← move right boundary LEFT
    }
    return lo;
}
// ═══ BINARY SEARCH ON ANSWER ═══
// Minimize x such that feasible(x) = true
int lo = minPossible, hi = maxPossible;
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (feasible(mid)) hi = mid;  // mid works, try smaller
    else lo = mid + 1;            // too small, go bigger
}
// lo = answer
```

💻 CH 23: CODING EXECUTION + DEBUGGING

CoderPad • Whiteboard • Clean code rules • Debugging protocol • Recovery when stuck

1 CODERPAD STRATEGY

- ✅ Keep imports at top: `import java.util.*;`
- ✅ Write a skeleton class + method signature first
- ✅ Use // comments as your coding plan before filling in
- ✅ Call helper methods before implementing them (top-down)
- ✅ Use `System.out.println()` for debug traces during dry run
- ❌ Don't use System.exit() or scanner — not needed
- ❌ Don't use static fields unless asked for design
- ❌ Don't rename variables halfway through (confusing)

💡 Write the function signature + guard clauses + helper stubs BEFORE any logic. It signals organized thinking.

2 CLEAN CODE CHECKLIST (10 RULES)

✓Return type and method name clearly describe purpose
✓First line: null/empty guard (`if (nums == null || nums.length == 0) return ...;`)
✓Descriptive variable names: `leftPtr` not `l`, `maxSoFar` not `m`
✓One responsibility per function — extract helpers for complex logic
✓Use `Math.max()`, `Math.min()` over verbose if-else
✓Use `Integer.compare(a, b)` in comparators — never `a - b` (overflow!)
✓Loop variable names match their domain: `row, col` not `i, j` for grids
✓Explicit base case for recursion at the top
✓Use `long` for sums that may overflow 32-bit int
✓Maximum 25 lines per method — refactor if longer

3 5-STEP RECOVERY WHEN STUCK

1. VERBALIZE: Explain what you know and what you're stuck on

↓

2. DRAW: Sketch a 3-4 element concrete example on paper/screen

↓

3. SIMPLIFY: "What if N=1? What if all elements are the same?"

↓

4. DATASTRUCTURE: "Can sorting / a HashMap / a queue help here?"

↓

5. ASK: "I'm thinking X or Y — does either direction seem promising to you?"

❌ NEVER sit silent for more than 90 seconds. Always verbalize your mental state, even if stuck!

4 DEBUGGING PROTOCOL — WHEN CODE FAILS

The 5-step debug process:

1. **Re-read the failing test case** — what is the expected output?
2. **Trace with print statements** — add System.out.println() for key variables
3. **Check loop bounds** — off-by-one errors are #1 source of bugs
4. **Check null/empty handling** — is the guard at the top correct?
5. **Check data structure initialization** — HashMap cleared? Arrays zeroed?

💡 **Say out loud:** "Let me trace through this specific failing case step by step..." — interviewers love seeing systematic debugging!

5 VARIABLE NAMING GUIDE

❌ Weak Name	✅ Strong Name	Context
l, r	leftPtr, rightPtr	Two Pointers
m	maxSoFar, maxLen	Sliding Window
i, j	row, col	Grid traversal
n	nodeCount, totalNodes	Graph
tmp	prevNode, nextNode	Linked List
h	maxHeap, minHeap	Priority Queue
dp	minCostDp, waysCount	DP table
vis	visitedNodes, seen	Graph DFS/BFS

🗣️ CH 24: COMMUNICATION MASTERY

Thinking out loud • Senior scripts • Handling hints • Trade-off language

1 SENIOR ENGINEER COMMUNICATION MATRIX — EXACT PHRASES

Interview Stage	L4 Says (❌ Avoid)	L5 Says (✅ Target)	L6 Says (🌟 Aim For)
Opening	"Let me just solve it."	"Before I start — a few clarifications about constraints..."	"Interesting problem — this reminds me of shortest path in a DAG. Let me verify: is the input always a valid DAG?"
Naming approach	"I'll use a map."	"I'll use a HashMap here because I need O(1) complement lookup — this brings time from $O(N^2)$ to O(N)."	"Two approaches: HashMap for O(N) average, or Two Pointers on sorted input for O(N) worst-case with O(1) space. Given no sort cost, HashMap is simpler."
Complexity	"It's O(N)."	"Time is O(N log K) because each of the N elements does at most one heap operation costing O(log K). Space is O(K) for the heap."	"Time O(N log K), space O(K). Note: if K ≈ N, heap approach degrades to O(N log N) — in that case, nth_element (QuickSelect) gives O(N) average with O(1) extra space."
Dry Run	"I think it works."	"Let me trace through my example: nums=[3,1,4,1], target=5. At i=0, complement=2, seen={} miss, store 3→0. At i=1, complement=4..."	"Let me construct an adversarial case: what if all elements are the same? nums=[3,3,3], target=6 — my code handles this correctly because each index is distinct in the map."
Receiving hints	"Oh OK." (continues same approach)	"That's a great point — you're suggesting the sorted input enables Two Pointers? Let me think about that: we could sort in O(N log N) then use O(N) two-pointer scan..."	"Yes, that edge case I hadn't considered — if the input contains Long.MAX_VALUE, my subtraction overflows. Let me fix the guard: use target - nums[i] computed in long first."

2 THINKING OUT LOUD — SCRIPT LIBRARY

When recognizing a pattern:

"This looks like a sliding window problem because we're asked for the longest contiguous subarray satisfying a condition — that's the classic signal."

When comparing approaches:

"I see two ways: sort + two pointers in O(N log N) time and O(1) space, or hash map in O(N) time and O(N) space. Given N is 10^5 and space isn't constrained, I'll go with the hash map for simplicity."

When writing a helper:

"I'll extract this into a helper `isValid(row, col)` to keep the DFS clean — I'll implement it right after."

When stuck:

"I know the brute force, but I'm trying to see if there's a way to avoid the inner loop. Does caching prefix sums help here?"

3 TRADE-OFF DISCUSSION FRAMEWORK

Always structure trade-offs as:

"Approach A gives [Time A] time and [Space A] space. Approach B gives [Time B] time and [Space B] space. Given [constraint from clarification], I'll choose [A or B] because [specific reason]."

Trade-off Dimension	What to Say
Time vs Space	"Trading O(N) space for O(1) better time — worth it here because N ≤ 10^5"
Worst vs Average	"QuickSelect is O(N) average but $O(N^2)$ worst — for guaranteed O(N log K) use heap"
Simplicity vs Optimality	"The simpler O(N log N) solution is bug-free and fast enough; QuickSelect saves a log factor but adds complexity"
Recursion vs Iteration	"Recursion is cleaner here. For very deep trees ($N=10^5$), stack overflow is a risk — iterative DFS with explicit stack is safer"

🎯 **COMMUNICATION FORMULA:** Clarify → Example → Brute Force → Optimal Insight → Get Agreement → Code → Dry Run → Complexity → Follow-up. *Never skip a step.*

🧪 MOCK 1: GOOGLE HARD — COURSE SCHEDULE II

Full 45-minute mock with evaluation rubric and score

PROBLEM

🔴 Hard 🔵 Google LC 210

There are numCourses courses (0 to numCourses-1). You are given prerequisites where prerequisites[i] = [a, b] means to take course a you must first take b. Return a valid course ordering. If impossible, return [].

CANDIDATE INTERVIEW EXECUTION

Candidate: "First, clarifications: Can there be duplicate edges? What is the max numCourses — 2000? Can courses have no prerequisites? What should I return if numCourses=0?"

Interviewer: "No duplicates, max 2000, return [] for 0 courses."

Candidate: "This is a topological sort problem on a DAG. I'll model each prerequisite [a,b] as a directed edge b→a (b before a). Then run Kahn's algorithm with in-degree counting. If all N courses are processed, return the order; if any course remains (cycle detected), return [].

Time: O(V+E), Space: O(V+E). Shall I code this?"

Interviewer: "Yes, go ahead."

💡 **AHA:** The trick is modeling prerequisites as directed edges correctly — b→a means b is prerequisite of a.

DRY RUN: numCourses=4, prerequisites=[[1,0],[2,0],[3,1],[3,2]]

Step	Queue	inDegree	order
Init	[0]	[0,1,1,2]	[]
Poll 0	[1,2]	[0,0,0,2]	[0]
Poll 1	[2]	[0,0,0,1]	[0,1]
Poll 2	[3]	[0,0,0,0]	[0,1,2]
Poll 3	[]	[0,0,0,0]	[0,1,2,3] ✅

COMPLETE JAVA SOLUTION

```java
public int[] findOrder(int numCourses, int[][] prerequisites) {
    // GUARD: edge case
    if (numCourses == 0) return new int[]{};

    // BUILD GRAPH + compute in-degrees
    List<List<Integer>> g = new ArrayList<>();
    int[] inDegree = new int[numCourses];
    for (int i = 0; i < numCourses; i++) g.add(new ArrayList<>());

    for (int[] prereq : prerequisites) {
        int course = prereq[0], pre = prereq[1];
        g.get(pre).add(course);   // edge: pre → course
        inDegree[course]++;
    }

    // INIT QUEUE with zero in-degree courses
    Queue<Integer> q = new ArrayDeque<>();
    for (int i = 0; i < numCourses; i++)
        if (inDegree[i] == 0) q.offer(i);

    // PROCESS: Kahn's BFS topological sort
    int[] order = new int[numCourses];
    int idx = 0;
    while (!q.isEmpty()) {
        int course = q.poll();
        order[idx++] = course;
        for (int next : g.get(course)) {
            if (--inDegree[next] == 0)
                q.offer(next);
        }
    }
    // If all courses processed → no cycle
    return idx == numCourses ? order : new int[]{};
}
// Time:  O(V + E) — V=numCourses, E=prerequisites.length
// Space: O(V + E) — adjacency list + in-degree array
```

FOLLOW-UP QUESTIONS

1. **"What if we need all possible orderings?"** — DFS backtracking, return all valid topological sorts
2. **"What if some courses are elective (no path from source)?"** — disconnected graph; still works since we init all zero-indegree nodes
3. **"If N = 10^6 courses, any issue?"** — Java recursion depth for DFS approach could overflow. Kahn's iterative BFS is safe.
4. **"How do you detect which specific cycle exists?"** — Color-based DFS: white/gray/black states; gray node re-visit = back edge = cycle

EVALUATION RUBRIC + SCORE

Dimension	Score (1-4)	Feedback
Problem Solving	4	Identified graph + topo sort in under 2 min
Code Quality	4	Clean, modular, proper naming, guard clause
Verification	4	Full step-by-step dry run with trace table
Communication	4	Narrated full 7-step framework
OVERALL	🌟 **STRONG HIRE (L5/L6)**	

🧪 MOCK 2: META MEDIUM + MOCK 3: AMAZON DESIGN

MOCK 2: META — MERGE INTERVALS (LC 56)

🟡 Medium 🟣 Meta

Given an array of intervals, merge all overlapping intervals.

Candidate: "Are intervals guaranteed non-negative? Can end < start exist? Is output order important?"
Interviewer: "Valid intervals, any order."
Candidate: "Sort by start time. Then single pass: if current interval's start ≤ last merged end, merge by taking max of ends. Otherwise, add as new interval. O(N log N) time, O(N) space for output."

```
public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) → Integer.compare(a[0], b[0]));
    List<int[]> merged = new ArrayList<>();
    for (int[] inv : intervals) {
        if (merged.isEmpty() || merged.get(merged.size()-1)[1] < inv[0])
            merged.add(inv);          // no overlap: add as-is
        else
            merged.get(merged.size()-1)[1] =   // overlap: extend end
                Math.max(merged.get(merged.size()-1)[1], inv[1]);
    }
    return merged.toArray(new int[merged.size()][]);
}
// Time: O(N log N)  Space: O(N)
```

DRY RUN: [[1,3],[2,6],[8,10],[15,18]]

inv	last.end	action	merged
[1,3]	—	add	[[1,3]]
[2,6]	3	2≤3 → extend to 6	[[1,6]]
[8,10]	6	8>6 → add	[[1,6],[8,10]]
[15,18]	10	15>10 → add	[[1,6],[8,10],[15,18]]

💡 Meta style: speed matters. Aim to code this in under 10 minutes with zero bugs.

MOCK 3: AMAZON — LRU CACHE (LC 146)

🟡 Medium 🟠 Amazon

Design an LRU cache with O(1) get and put operations.

Candidate: "Key insight: O(1) get + put means I need a data structure that supports O(1) ordered access. HashMap + Doubly Linked List: HashMap for O(1) key lookup, DLL for O(1) order maintenance. Sentinel head/tail nodes avoid null checks."

```
class LRUCache {
    class Node { int key, val; Node prev, next;
        Node(int k, int v){ key=k; val=v; } }
    private final Map<Integer,Node> map = new HashMap<>();
    private final Node head = new Node(0,0), tail = new Node(0,0);
    private final int cap;
    public LRUCache(int capacity) {
        this.cap = capacity;
        head.next = tail; tail.prev = head;  // sentinel setup
    }
    public int get(int key) {
        if (!map.containsKey(key)) return -1;
        Node node = map.get(key);
        remove(node); insertFront(node);   // move to MRU position
        return node.val;
    }
    public void put(int key, int value) {
        if (map.containsKey(key)) remove(map.get(key));
        if (map.size() == cap) {   // evict LRU (tail.prev)
            map.remove(tail.prev.key); remove(tail.prev); }
        Node node = new Node(key, value);
        insertFront(node); map.put(key, node);
    }
    private void remove(Node n) { n.prev.next=n.next; n.next.prev=n.prev; }
    private void insertFront(Node n) {
        n.next=head.next; n.next.prev=n; head.next=n; n.prev=head; }
}
// ALL ops: O(1) time  |  Space: O(capacity)
```

💡 Amazon LP integration: "This is a classic ownership problem — one oversight in the eviction logic (wrong pointer) causes production cache corruption."

🧪 MOCK 4: UBER HARD + MOCK 5: STAFF-LEVEL

MOCK 4: UBER — N-QUEENS (LC 51)

🔴 Hard 🚙 Uber

Place N queens on N×N board so none attacks another. Return all valid configurations.

Candidate: "Classic backtracking. Each row gets exactly one queen. For each row, try each column — skip if column, diagonal, or anti-diagonal already occupied. Use three HashSets for O(1) conflict checking. Choose → place → recurse → undo."

```
public List<List<String>> solveNQueens(int n) {
    List<List<String>> res = new ArrayList<>();
    int[] queens = new int[n]; // queens[row] = col
    Arrays.fill(queens, -1);
    Set<Integer> cols = new HashSet<>(),
        diag = new HashSet<>(),    // row - col
        antiDiag = new HashSet<>(); // row + col
    backtrack(0, n, queens, cols, diag, antiDiag, res);
    return res;
}
void backtrack(int row, int n, int[] queens,
    Set<Integer> cols, Set<Integer> diag, Set<Integer> antiDiag,
    List<List<String>> res) {
    if (row == n) { res.add(buildBoard(queens, n)); return; }
    for (int col = 0; col < n; col++) {
        if (cols.contains(col) || diag.contains(row-col)
            || antiDiag.contains(row+col)) continue;
        // CHOOSE
        queens[row] = col;
        cols.add(col); diag.add(row-col); antiDiag.add(row+col);
        // EXPLORE
        backtrack(row+1, n, queens, cols, diag, antiDiag, res);
        // UNDO (state restoration)
        queens[row] = -1;
        cols.remove(col); diag.remove(row-col); antiDiag.remove(row+col);
    }
}
// Time: O(N!) worst  Space: O(N) recursion depth
```

FOLLOW-UPS:

- "Count unique solutions only?" → just increment counter, not build board
- "Solve for N=100?" → not feasible with backtracking; use optimization (branch-and-bound, symmetry)
- "How is this used at Uber?" → dispatch routing = constraint satisfaction at scale

MOCK 5: STAFF LEVEL — FIND MEDIAN FROM DATA STREAM (LC 295)

🔴 Hard 👑 Staff Level

Design a MedianFinder that supports addNum(int num) and findMedian() in O(log N) and O(1) respectively.

Candidate: "Two heaps: max-heap for lower half, min-heap for upper half. Invariant: maxHeap.size() == minHeap.size() or maxHeap.size() == minHeap.size()+1. Median = maxHeap.peek() if odd, or average of both tops if even. Staff-level extension: if data is mostly skewed to one side, discuss memory-efficient approaches using reservoir sampling."

```
class MedianFinder {
    // maxHeap holds lower half (negated for max behavior)
    private final PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Col
lections.reverseOrder());
    // minHeap holds upper half
    private final PriorityQueue<Integer> minHeap = new PriorityQueue<>();

    public void addNum(int num) {
        maxHeap.offer(num);              // 1. always add to maxHeap first
        minHeap.offer(maxHeap.poll());   // 2. balance: push max of lower
to upper
        if (maxHeap.size() < minHeap.size())     // 3. maintain size invar
iant
            maxHeap.offer(minHeap.poll());
    }
    public double findMedian() {
        if (maxHeap.size() > minHeap.size())
            return maxHeap.peek();       // odd total: lower half has one
more
        return (maxHeap.peek() + minHeap.peek()) / 2.0; // even: average o
f midpoints
    }
}
// addNum: O(log N)  findMedian: O(1)  Space: O(N)
```

STAFF-LEVEL EXTENSIONS

1. **99% data within range [0,100]?** → Use integer frequency count array, O(1) add, O(1) median
2. **Remove elements?** → Lazy deletion with HashSet of "removed" keys, recalculate on access
3. **Sliding window median?** → Two TreeMaps with frequency counts (LC 480)

🏢 CH 27: COMPANY GUIDES — GOOGLE · META · AMAZON · MICROSOFT

🔵 GOOGLE — Interview Guide

Google Interview Profile

Rounds	5–6 rounds: 4-5 coding + 1 system design (Sr+)
Difficulty	Hard. Expect unseen variant problems, not LeetCode exact copies
Top Patterns	Graph BFS/DFS, Trees, DP (intervals, 2D), Topo Sort, Segment Tree
Style	Mathematical correctness over speed; expect proofs for greedy choices
Communication	Think out loud constantly; they evaluate process as much as solution
Common Mistakes	Solving easy version without handling edge cases; not proving correctness
Preparation	NeetCode Hard problems; graph variants; BST edge cases; practice verbal explanation
Time Management	40min coding + 5min follow-up; never run over without checking time

💡 Google cares deeply about correctness and mathematical rigor. If you choose greedy, you should be able to explain the exchange argument.

🟣 META — Interview Guide

Meta Interview Profile

Rounds	2 coding rounds (45 min each) — often 2 problems per round!
Difficulty	Medium-Hard but known problems; speed is crucial
Top Patterns	Arrays, Strings, Two Pointers, BFS/DFS, Trees, HashMap, Sliding Window
Style	Velocity. They expect clean, bug-free code quickly (aim 15 min per problem)
Communication	Brief clarification, then code fast. Less conversation than Google.
Common Mistakes	Too much talking; running out of time on second problem
Preparation	Blind 75 + Meta-tagged LeetCode; practice writing clean code in <15 min
Behavioral	Meta Leadership Principles: Move fast, Be bold, Focus on impact

💡 Meta = speed + accuracy. Practice coding common patterns in under 12 minutes, bug-free.

🟠 AMAZON — Interview Guide

Amazon Interview Profile

Rounds	4–5 rounds: coding + LP behavioral in every single round
Difficulty	Medium. Breadth over depth — practical DS problems
Top Patterns	Heap/PQ, HashMap, BFS/DFS, Sliding Window, Design (LRU, LFU)
Style	LP integration every round; always have STAR stories ready
Communication	Frame technical decisions using Leadership Principles language
Common Mistakes	Ignoring LP questions; not connecting code to real-world impact
Preparation	Map all 16 LPs to your STAR stories; practice out loud
LP Focus	Bias for Action, Customer Obsession, Dive Deep, Ownership, Frugality

💡 Amazon = Coding + Storytelling. Prepare 3-5 rich STAR stories that map to multiple LPs.

🟢 MICROSOFT — Interview Guide

Microsoft Interview Profile

Rounds	4–5 rounds: coding + behavioral + hiring manager final
Difficulty	Easy-Medium. Strong emphasis on code clarity and testing
Top Patterns	Linked List, Trees (BST), Stack, Arrays, String, Basic DP
Style	Code quality + verbal explanation + "How would you test this?"
Communication	Collaborative; interviewer may guide you more than at Google
Common Mistakes	Not discussing testing strategy; ignoring null edge cases
Preparation	LeetCode Easy-Medium; practice explaining code after writing
Special	"How would you test this function?" — always prepare an answer

💡 Microsoft rewards thoroughness. Voluntarily discuss testing strategy and null handling.

🏢 COMPANY GUIDES — UBER · STRIPE · NETFLIX · DATADOG · AIRBNB

🚗 UBER

Rounds	4 rounds: coding, SD, behavioral, HM
Top Patterns	Graphs (Dijkstra!), Geospatial, Concurrency, Heap
Style	Practical, production-ready; expect large N problems
Special	Geospatial indexing, real-time matching algorithms
Prep Tip	Master Dijkstra, geohash concepts, surge pricing algorithms

💳 STRIPE

Rounds	4 rounds: coding in real IDE (not CoderPad)
Top Patterns	Practical parsing, String manipulation, System design
Style	Real IDE + real tests; expected to write unit tests!
Special	JSON parsing, rate limiting, idempotency key design
Prep Tip	Practice writing clean, testable Java in your IDE, not online editors

📺 NETFLIX

Rounds	3-4 rounds; strong system design focus at senior
Top Patterns	Data pipeline, Streaming algorithms, LRU/Cache design
Style	"Freedom and Responsibility" culture; ownership is key
Special	Chaos engineering mindset; resilience + failure handling
Prep Tip	Read Netflix Tech Blog; understand their chaos engineering

📊 DATADOG

Rounds	4 rounds: coding + metrics system design
Top Patterns	Sliding Window, Time Series, Heap, HashMap, Rate limiting
Style	Observability-focused; expect questions about metrics at scale
Special	Moving average, percentile calculation (P99), anomaly detection
Prep Tip	Understand time-series data structures, circular buffers, rolling windows

💡 Datadog loves questions about "how would you compute P99 latency efficiently from a stream?"

🏠 AIRBNB

Rounds	4 rounds: coding + infra design + cross-functional interview
Top Patterns	Intervals, Calendar scheduling, Booking availability, Graph traversal
Style	Practical domain-specific questions (booking, calendar, pricing)
Special	Merge intervals with availability calendars; conflict detection
Prep Tip	Master interval problems; practice booking system design

💡 Airbnb loves: "Given N bookings, find all available time slots between check-ins."

COMPANY COMPARISON MATRIX

Company	Speed	Difficulty	Behavioral	System Design	Top Skill to Practice
🔵 Google	Medium	⭐⭐⭐⭐⭐	Moderate	Sr+ required	Graph algorithms, algorithmic correctness
🟣 Meta	🔥 Fast	⭐⭐⭐⭐	Light	Sr+ required	Speed + accuracy, no time waste
🟠 Amazon	Medium	⭐⭐⭐	🔥 Heavy (LP)	Sr+ required	STAR stories + practical DS
🟢 Microsoft	Relaxed	⭐⭐⭐	Moderate	Optional	Code quality + testing strategy
🚗 Uber	Medium	⭐⭐⭐⭐	Light	Sr+ required	Dijkstra, spatial indexing
💳 Stripe	Medium	⭐⭐⭐	Moderate	Sr required	Clean code + unit testing in IDE

📜 CH 28: REVISION HANDBOOK + STUDY PLANS

Pattern matrix • Blind 75 map • NeetCode 150 map • Last month/week/day plans

ULTIMATE COMPLEXITY REVISION MATRIX — 20 CORE PATTERNS

Pattern	Time	Space	DS Used	When to Use	Top FAANG Problems
Two Pointers	O(N)	O(1)	Two int pointers	Sorted array, pair/triple sum	LC 15, 11, 42, 125
Sliding Window	O(N)	O(1)/O(K)	int[128] freq, l/r	Contiguous subarray/string	LC 3, 76, 424, 239
Binary Search	O(log N)	O(1)	lo, hi, mid	Sorted space or BS on answer	LC 33, 875, 410, 4
Prefix Sum	O(N)	O(N)	int[] prefix + HashMap	Subarray sum queries	LC 560, 525, 1248
Monotonic Stack	O(N)	O(N)	Deque<Integer>	Next greater/smaller element	LC 739, 84, 85, 42
Top-K Heap	O(N log K)	O(K)	PriorityQueue	K largest/smallest/frequent	LC 347, 295, 215, 973
Tree DFS	O(N)	O(H)	Call stack	Path, depth, subtree problems	LC 104, 236, 124, 543
Tree BFS	O(N)	O(W)	Queue<TreeNode>	Level-order, row traversal	LC 102, 199, 103
Graph DFS	O(V+E)	O(V)	boolean[] visited	Flood fill, components	LC 200, 130, 695, 417
Graph BFS	O(V+E)	O(V)	Queue + visited	Shortest path unweighted	LC 994, 1162, 127
Topo Sort	O(V+E)	O(V)	int[] indegree + Queue	DAG ordering, dependencies	LC 207, 210, 269
Dijkstra	O(E log V)	O(V+E)	PriorityQueue<int[]>	Shortest path weighted	LC 743, 1514, 787
Union-Find	O(α(N))	O(N)	int[] parent, rank	Grouping, cycle detect	LC 547, 684, 1319
1D DP	O(N)	O(N)→O(1)	int[] dp	Min cost, count ways, LIS	LC 70, 198, 300, 322
2D DP	O(NM)	O(NM)→O(M)	int[][] dp	Grid, string match, knapsack	LC 62, 1143, 72, 416
Backtracking	$O(2^N)$/O(N!)	O(N)	List path + recursion	All combos, permutations	LC 78, 39, 46, 51
Greedy + Sort	O(N log N)	O(1)/O(N)	Arrays.sort	Optimal local → global	LC 55, 134, 621, 406
Intervals	O(N log N)	O(N)	Sort by start + scan	Merge, insert, overlap	LC 56, 57, 435, 253
Trie	O(L)	O(N·L)	TrieNode[26]	Prefix, word search, autocomplete	LC 208, 211, 212, 421
Bit Manipulation	O(1)/O(N)	O(1)	int bitmask	XOR tricks, subsets, masking	LC 136, 191, 338, 78

LAST MONTH PLAN (4 WEEKS)

Week	Focus	Problems/Day
Week 1	Arrays, Strings, HashMap, Two Pointers, Sliding Window	3 Medium
Week 2	Trees, BST, Heap, Binary Search, Linked List	3 Medium
Week 3	Graphs, DP (1D/2D), Backtracking, Greedy, Intervals	2 Medium + 1 Hard
Week 4	Mock interviews, revision, weak areas, system design	Full mocks

LAST WEEK PLAN (7 DAYS)

Day	Focus
Day 1	Arrays, HashMap, Two Pointers top 10
Day 2	Trees: DFS + BFS + BST top 10
Day 3	Graphs: BFS/DFS/Topo/Dijkstra top 10
Day 4	DP: 1D, 2D, Knapsack top 10
Day 5	Heap, Binary Search, Backtracking top 10
Day 6	Full mock interview (timed, 45 min)
Day 7	Review mistakes, cheat sheets only — rest!

LAST DAY PLAN (24 HRS)

✓Review pattern cheat sheet — 30 min
✓Review complexity table above
✓Practice 3 favourite templates from memory
✓Review one mock interview out loud
✓Prepare 3 LP stories for Amazon/behavioral
✓Sleep 8 hours — seriously!
✓Light exercise in the morning
❌Do NOT solve new hard problems
❌Do NOT stay up late "just in case"

💎 CH 29: ULTIMATE INTERVIEW PLAYBOOK + OFFER NEGOTIATION

Before • During • After • If stuck • Offer strategy

✅ BEFORE INTERVIEW CHECKLIST

Night Before:

✓Review pattern cheat sheet (20 min max)
✓Prepare IDE / CoderPad snippet (import java.util.*;)
✓Review your STAR stories for LP questions
✓Sleep 8 hours — peak cognitive performance requires sleep
❌Don't solve new hard problems night before

Morning Of:

✓Light exercise (30 min walk)
✓Good breakfast, no caffeine overload
✓Join 5 min early, test audio/video
✓Have water and scratch paper ready
✓Remind yourself: this is a conversation, not an exam

💻 DURING INTERVIEW PROTOCOL

0-3 min: Read problem. NEVER code yet.

↓

Ask clarifying questions about constraints

↓

Work through a small concrete example

↓

State brute force + why it's suboptimal

↓

Name optimal approach + get agreement

↓

Code: Guard → Helper stubs → Logic → Return

↓

Dry run your code on the example

↓

State complexity (time + space) + edge cases

↓

Volunteer follow-up: "A harder version would be..."

🆘 IF COMPLETELY STUCK

Say out loud: "Let me think step by step about what I know..."

Strategy 1: Solve a simpler version — "What if N=1?"
Strategy 2: Draw a concrete example — visualize the state machine
Strategy 3: Try brute force out loud — it may reveal the optimization
Strategy 4: "I'm thinking of X or Y — does either feel right to you?"
Strategy 5: "What if I sort first / use a HashMap / use a queue?"

💡 **Interviewers can HELP you if you verbalize where you're stuck!** Silent struggling = unsalvageable.

If you take a hint:

Say: "That's a great point — let me think about how that changes the approach. If I use [hint], then I can avoid [bottleneck] because..."

Taking a hint gracefully = demonstrates intellectual humility = good signal!

💰 OFFER NEGOTIATION — 4 GOLDEN LAWS

Law 1: Never Give First Number
"I'd love to understand the full compensation structure first — what's the range for this level?"

Law 2: Leverage Competing Offers
"I have an offer from [Company] for $X TC. I'm more excited about [your company] — can you match or improve?"

Law 3: Negotiate TC Not Salary
Focus on RSU cliff, vesting schedule, sign-on bonus — base salary cap is harder to move than equity/sign-on.

Law 4: Anchor with Enthusiasm
"I'm very excited about this role. If we can reach $X TC total, I'll sign immediately — I'm not shopping around."

💡 **Script:** "I'm very excited about the opportunity. The offer is $Y — I was expecting around $X based on my research and the competing offer. Is there any flexibility in the RSU grant?"

✅ FINAL INTERVIEW MASTER CHECKLIST

Before coding (must do all 5):

✓Clarify input type, constraints, N range
✓Work through a concrete example
✓Name brute force + explain why it's slow
✓Pitch optimal approach + get agreement
✓State time/space complexity before coding

While coding (must do all 5):

✓Guard clause at top (null, empty check)
✓Descriptive variable names throughout
✓Think out loud while writing every loop
✓Extract helpers for complex sub-problems
✓Use Integer.compare() in all comparators

After coding (must do all 5):

✓Dry run with your own concrete example
✓Test edge cases: empty, single, duplicates
✓State final time and space complexity
✓Volunteer follow-up extension unprompted
✓Ask a thoughtful question about the team

🧠 THE FAANG LEARNING FRAMEWORK — OUR UNIQUE TEACHING SYSTEM

13 phases • Mentor style principles • Signature workflow • Template-first learning

THE 13-PHASE FAANG LEARNING FRAMEWORK (Applied to EVERY Topic)

Phase	Name	Goal	Key Activity
Phase 0	🎯 Mindset	WHY does this exist?	Why do companies ask this? What does it solve? Interview frequency?
Phase 1	💡 Intuition First	Build the feeling	Story → Real-world analogy → Draw it → Build from scratch → AHA moment
Phase 2	📖 Core Understanding	Lock in definitions	Formal definitions → Properties → Rules → Edge cases → Common confusions
Phase 3	🎯 Pattern Recognition	Brain learns to spot it	Recognition keywords → When NOT to use → Similar pattern comparison
Phase 4	🔨 Build Algorithm Together	Derive, don't memorize	Brute Force → Identify bottleneck → Optimize step by step → Why it works
Phase 5	⚙️ Template Building	Create reusable code	Generic template → Java template → Interview skeleton → Memory tricks
Phase 6	▶️ Dry Run	See variables move	Small example → Medium example → Visual trace → State change table
Phase 7	💻 Coding Together	One line at a time	Explain every variable → Every loop → Every condition → Complete Java code
Phase 8	📊 Complexity Thinking	Analyze rigorously	Time O() → Space O() → Why? → Can it be improved? → Trade-offs
Phase 9	🐛 Debugging Mindset	Find and fix bugs	Common bugs → Edge cases → Dry-run mistakes → Debug checklist
Phase 10	🔄 Reverse Thinking	Handle follow-ups	Constraint changes → Solution changes → Alternative approaches → Why not X?
Phase 11	📈 Pattern Expansion	Generalize mastery	Easy → Medium → Hard → Similar pattern → Mixed pattern problems
Phase 12	✅ Mastery	Own it completely	Explain without notes → Draw from memory → Write template → Teach it back

🧠 OUR MENTOR STYLE PRINCIPLES

- 🚫 **Never memorize** — Derive the algorithm from first principles
- 🚫 **Never jump to code** — Build intuition first, always
- 📖 **Every algorithm has a story** — Dijkstra as GPS, BFS as ripple in water
- 💡 **Every topic has ONE Aha Moment** — the mental shift that unlocks understanding
- 🖍️ **Draw before coding** — visualize nodes, pointers, windows
- 🔨 **Build brute force first** — understand the problem before optimizing
- 📐 **Template-first** — Intuition → Algorithm → Template → Java code
- 🎓 **Master the pattern, not the problem** — solve 100 new problems from 15 templates

🔄 OUR SIGNATURE WORKFLOW

📖 Read Problem Carefully
↓
🔍 Find Clues (Keywords + N constraints)
↓
🎯 Recognize Pattern Family
↓
📋 Recall Generic Template
↓
✂️ Customize Template for This Problem
↓
▶️ Dry Run with Small Example
↓
💻 Write Clean Java Code
↓
📊 Analyze Time + Space Complexity
↓
🧪 Test Edge Cases
↓
🚀 Optimize + Follow-ups

🌳 TREE TEMPLATES (8 Total)

#	Template	Key Insight
T1	**DFS Preorder**	Root first → Copy/Serialize
T2	**DFS Inorder**	Left→Root→Right → BST sorted order
T3	**DFS Postorder**	Leaves-up → Height/Diameter
T4	**BFS Level Order**	Freeze level size, Queue
T5	**Bottom-Up Height**	1 + max(L, R)
T6	**BST Search/Validate**	min/max bounds recursion
T7	**LCA**	Return node if == p or q
T8	**Serialize/Deserialize**	Preorder + null markers

💡 Teaching flow: Draw → Parent/Child → Recursive Faith → Dry Run Stack → Template → Customize

🌐 GRAPH TEMPLATES (11 Total)

#	Template	Key DS
G1	**Build Undirected**	List<List<Integer>>
G2	**Build Weighted**	List<List<int[]>>
G3	**DFS (recursive)**	boolean[] visited
G4	**BFS shortest path**	Queue + dist[]
G5	**Multi-Source BFS**	All sources in Queue init
G6	**Grid DFS**	DIR4 + isValid()
G7	**Grid BFS**	Queue + visited[][]
G8	**Topological Sort**	indegree[] + Queue
G9	**Union-Find (DSU)**	parent[] + rank[]
G10	**Dijkstra**	PQ<int[]> + dist[]
G11	**Kruskal MST**	Sort edges + DSU

💡 AHA: Grids = Graphs! Trees ⊂ Graphs! Most graph problems hide as word problems.

🎄 TRIE TEMPLATES (9 Total)

#	Template	Key Insight
Tr1	**TrieNode Design**	TrieNode[26] + isEnd boolean
Tr2	**Insert**	Create nodes char by char
Tr3	**Search (exact)**	Traverse + check isEnd
Tr4	**StartsWith**	Traverse + return true (no isEnd check)
Tr5	**Prefix Count**	countChildren at prefix end
Tr6	**Word Search II**	Trie + DFS board backtrack
Tr7	**MapSum**	TrieNode with int val
Tr8	**Max XOR (Binary Trie)**	Bit-by-bit 31→0
Tr9	**Delete**	Postorder cleanup

💡 Teaching flow: String → Characters → One char = One node → Build path → End marker → Insert → Search

🎯 **THE PHILOSOPHY:** Master the 13-phase FAANG Learning Framework for every pattern. Derive the algorithm from first principles. Build the template. Write the code. The goal is to solve *any* unseen problem — not to memorize 300 solutions. **Master the pattern. Master any problem.**